

Final Assignment (Major Project): Custom Vulnerable Lab

Student Name: Ankan Sarkar

Role: Ethical Hacking & Cybersecurity Intern

Organization: VaultofCodes

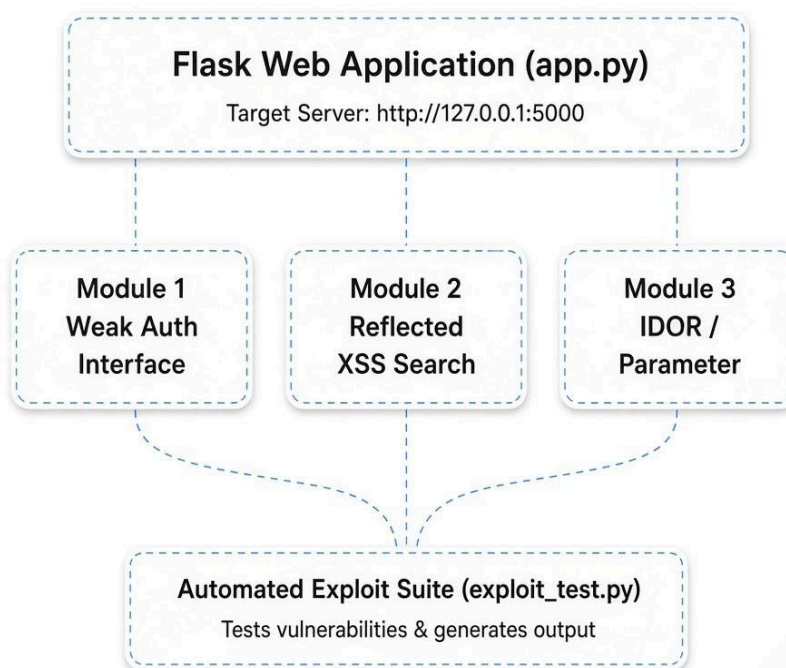
Program Duration: 1 Month

Problem Statement:

Modern web applications frequently suffer from critical security flaws due to insufficient input validation, weak credential management, and flawed access control logic. Security professionals and developers require controlled, safe laboratory environments to demonstrate how vulnerability chains operate in real time.

How Your Tool Works (Logic & Architecture Flow):

The system consists of two primary components:



1. **Weak Authentication Mechanism:** The `/login` endpoint checks submitted credentials against plain-text default combinations (`admin / admin123`) without implementing account lockout thresholds or multi-factor authentication.

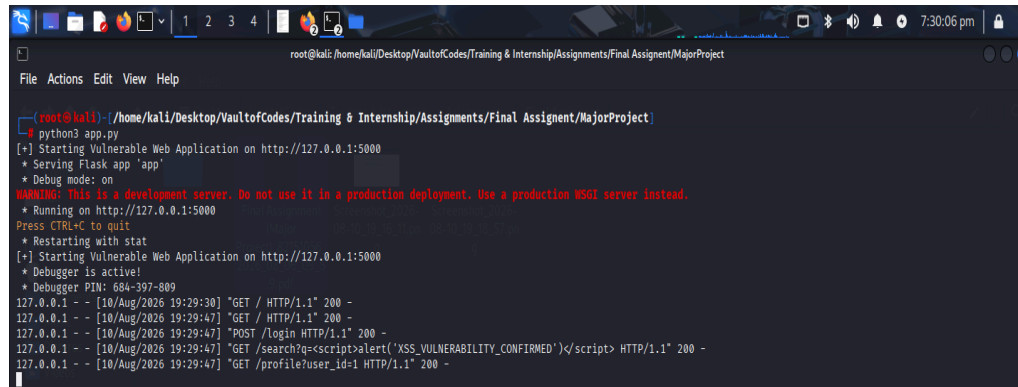
2. **Reflected XSS Engine:** The `/search?q=` parameter processes user input and reflects it dynamically into the HTML Document Object Model (DOM) without applying HTML entity sanitization (`htmlspecialchars`), enabling script execution.
3. **IDOR Access Control Logic:** The `/profile?user_id=` endpoint retrieves sensitive account records based solely on the incoming URL parameter without checking whether the requesting user's active session matches the requested profile ID.
4. **Exploit Verification Engine:** The standalone client `exploit_test.py` programmatically issues crafted HTTP POST and GET requests to verify exploit success against all three application endpoints.

Key Features Implemented:

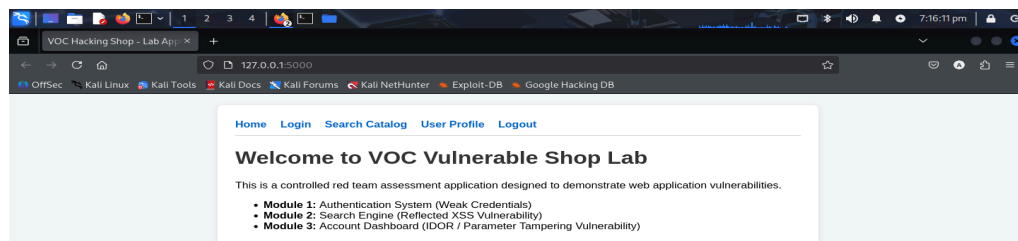
- **Multi-Vulnerability Lab Web Application:** Includes three security vulnerabilities (Weak Auth, Reflected XSS, IDOR/Parameter Tampering).
- **Responsive Control Panel & Navigation:** Integrated web interface rendering real-time profile data, search queries, and alert boxes.
- **Automated Exploit & Verification Script:** Independent Python testing script utilizing `requests` to test and report vulnerability statuses.
- **Data Extraction Demonstration:** Capability to extract private administrator API keys (`SECRET_API_KEY_ADMIN_9901`) via IDOR tampering.
- **Modular Code Structure:** Easily extensible Python codebase for adding future vulnerability modules (e.g., SQL Injection, CSRF).

Output & Execution Results:

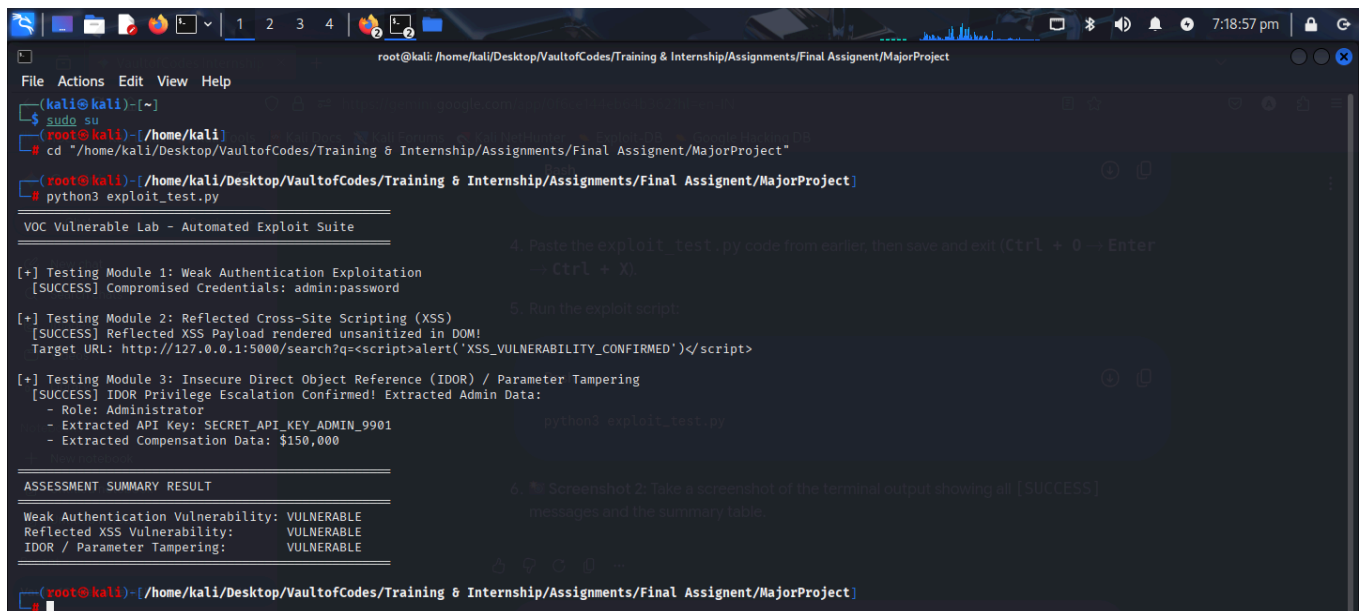
1. Server Launch Execution (**app.py**)



```
(root@kali) ~/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject
python3 app.py
[+] Starting Vulnerable Web Application on http://127.0.0.1:5000
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
[+] Starting Vulnerable Web Application on http://127.0.0.1:5000
* Debugger is active!
* Debugger PIN: 684-397-809
127.0.0.1 - - [10/Aug/2026 19:29:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "GET /search?q=<script>alert('XSS_VULNERABILITY_CONFIRMED')</script> HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "GET /profile?user_id=1 HTTP/1.1" 200 -
```



2. Automated Exploit Verification Output (**exploit_test.py**)



```
(kali@kali) [~]
$ sudo su
(root@kali) [/home/kali]
# cd "/home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject"
(root@kali) [/home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject]
# python3 exploit_test.py

VOC Vulnerable Lab - Automated Exploit Suite

[+] Testing Module 1: Weak Authentication Exploitation
[SUCCESS] Compromised Credentials: admin:password

[+] Testing Module 2: Reflected Cross-Site Scripting (XSS)
[SUCCESS] Reflected XSS Payload rendered unsanitized in DOM!
Target URL: http://127.0.0.1:5000/search?q=<script>alert('XSS_VULNERABILITY_CONFIRMED')</script>

[+] Testing Module 3: Insecure Direct Object Reference (IDOR) / Parameter Tampering
[SUCCESS] IDOR Privilege Escalation Confirmed! Extracted Admin Data:
  - Role: Administrator
  - Extracted API Key: SECRET_API_KEY_ADMIN_9901
  - Extracted Compensation Data: $150,000

ASSESSMENT SUMMARY RESULT

Weak Authentication Vulnerability: VULNERABLE
Reflected XSS Vulnerability: VULNERABLE
IDOR / Parameter Tampering: VULNERABLE

(root@kali) [/home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject]
```

Limitations

- **Local Environment Bound:** Designed specifically for localhost deployment (`127.0.0.1:5000`) without persistent database storage (uses in-memory Python dictionaries).
- **Simplified Access Control Matrix:** Does not currently simulate complex JWT token manipulation or multi-tenant database row-level security policies.
- **Single-Threaded Exploit Engine:** The verification script executes synchronous sequential HTTP requests rather than asynchronous multi-threaded brute-forcing.

Future Improvements:

- **Persistent SQL Database Integration:** Connect the backend to a PostgreSQL/SQLite database to demonstrate SQL Injection (SQLi) vectors.
- **Interactive Remediation Toggle:** Add a "Secure Mode" switch in the web UI that enables `htmlspecialchars()`, server-side session checks, and rate-limiting to allow side-by-side comparison of vulnerable vs. patched code.
- **Web-Based Dashboard Interface:** Upgrade the verification script into an interactive Streamlit UI showing real-time exploit HTTP request/response inspection.